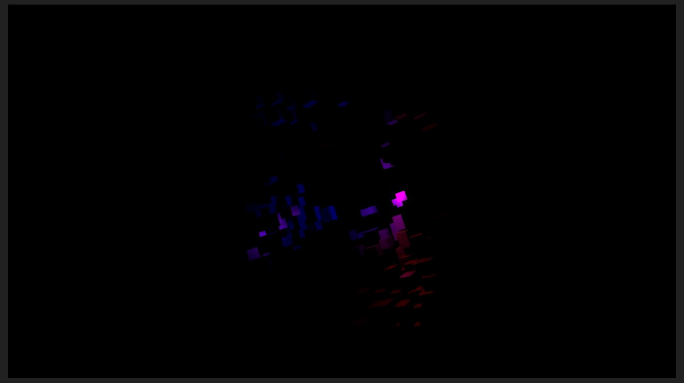
Real-Time Computer Graphics: Pictorial

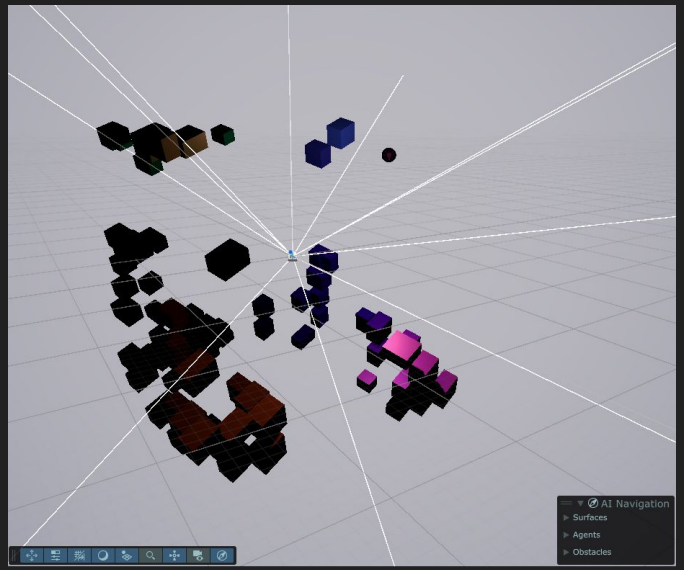
Abstract
In this pictorial nine exercises are presented and briefly explained in regards to the real-time computer graphics course. The exercises revolved around: **(1)** getting to know a game engine of choice, **(2)** creating a 3D model using photogrammetry, **(3)** using lighting to create a realistic scene, **(4)** Unity Shader Graph to make something cool, **(5)** camera effects to enhance the realism together with lighting, **(6)** high-level shader language (HLSL) if something specific is need that the Shader Graph cannot provide, **(7)** real-time animations for giving characters life, **(8)** a tiny virtual production to make a real life prop come to life, and finally **(9)** analysis of 3D objects for understanding a complex mesh or large point cloud. These exercises were made using the softwares Unity, RealityScan, PostShot, Cascadeur and Python.

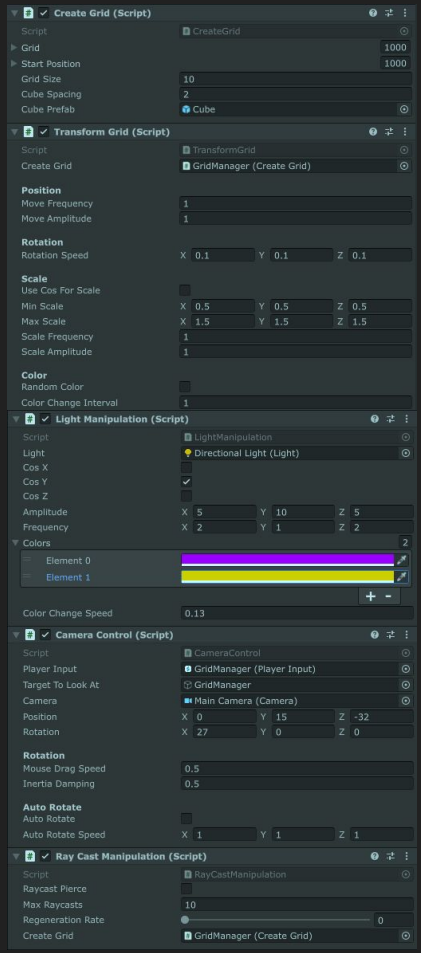
Keywords:
Real-Time Computer Graphics, Unity, Photogrammetry, Gaussian Splatting, 3D model, Lighting Effects, Shader Graph, Camera Effects, HLSL, Animation, Virtual Production, Python

Exercise 1: Instantiating Cubes

This exercise served as an introduction or refresher to the engine of choice, which for this project was Unity 6.2 (version 6000.2.10f1) using the Universal Render Pipeline. The task was to make a grid of cubes that would first be instantiated, then translated, rotated, scaled, and change color, along with changes to the lighting and camera. The cubes were first made as prefabs with a metallic material with a smooth surface so it would reflect light. As cubes are instantiated, they are stored in a List, from where their transform and color is manipulated. From the middle of the grid a Ray is cast towards a random point in a sphere, where it will disable the gameobject of a cube. They will then regenerate over time and can only be disabled again once the grid has fully regenerated. In order for the effect to “pop” the skybox was set to black and the directional light was removed. The only light source would move around in the middle of the grid structure in a sine or cosine pattern. To control the desired effect, multiple variables were serialized.



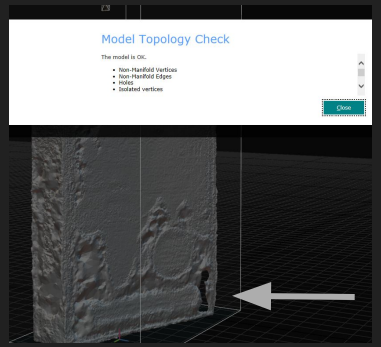




Exercise 2: Capturing Reality

Photogrammetry

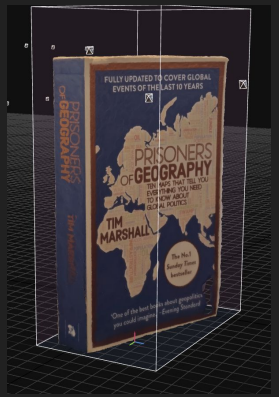
Initially for this exercise, the RealityScan mobile application was used to convert images (n = 235) of a real-world target, a book in this case, into a virtual 3D model. The results were not satisfactory, which were likely due to subpar image and lighting quality, resulting in a poor point cloud generation. Therefore, the desktop version of RealityScan (version 2.0.1) was used instead, with the same pictures. This yielded better results, likely due to the superior processing power available. Although the desktop results were superior to the mobile, the model still suffered imperfections in the generated topology in the form of a bumpy uneven surface and small holes. RealityScan was unable to perceive these issues with the "Model Topology Check" function, resulting in a suboptimal 3D model. Multiple attempts were made to achieve greater results, however these were the best results I could achieve with the available resources.



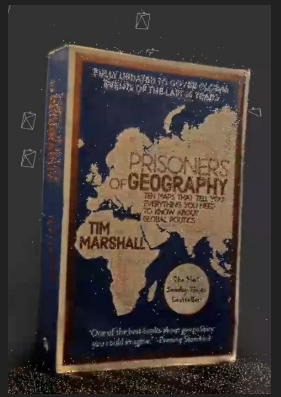
The model has a big hole but is unable to be fixed by the software

Gaussian Splatting

As to compare to the photogrammetry method provided by RealityScan, a Gaussian splat was trained using PostShot. The results came out somewhat faded and generally lacked more cohesion compared to that of the photogrammetry method, based on the available images.



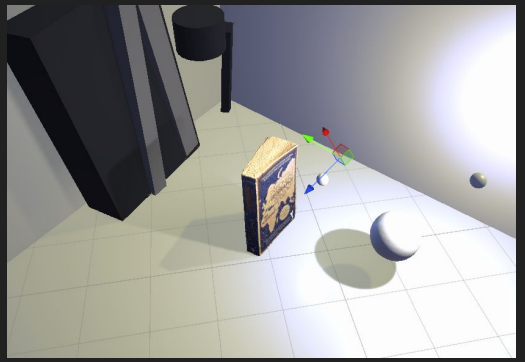
Photogrammetry



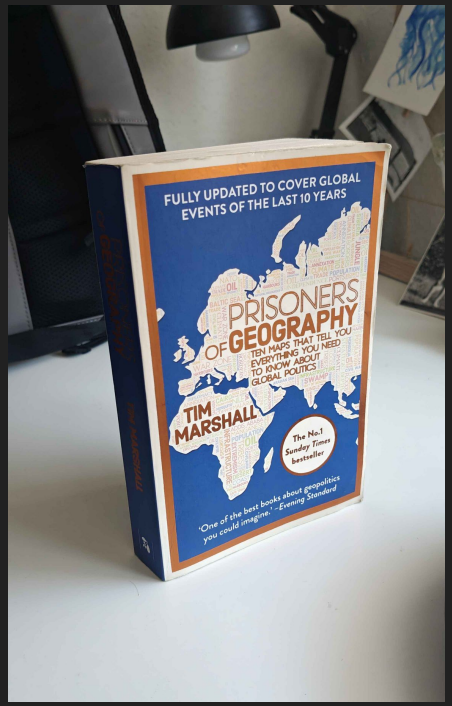
Gaussian splat

Exercise 3: Computer Graphics and Lighting Effects

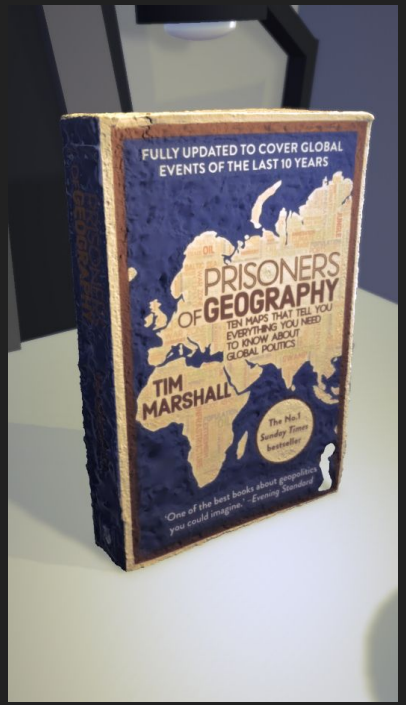
With the 3D model exported as an .obj file along with the relevant textures, it could be imported into Unity where I would attempt to recreate a simple photo I took of my real-world subject. The lighting of the scene consists of a directional light (the gizmo in the picture below) and two point lights - one blue and one grey colored light, each representing the blue sky and overcast sun respectively. I also played around with the values in the Volume component, such as the white balancing to get the desired look of the scene. The scene and background elements are made from simple meshes found in Unity (cubes, spheres, cylinders, and planes).



The setup of the scene with lighting and background elements as seen from another angle. The two point lights are the smaller spheres, while the bigger one provides the shadow in the bottom right corner



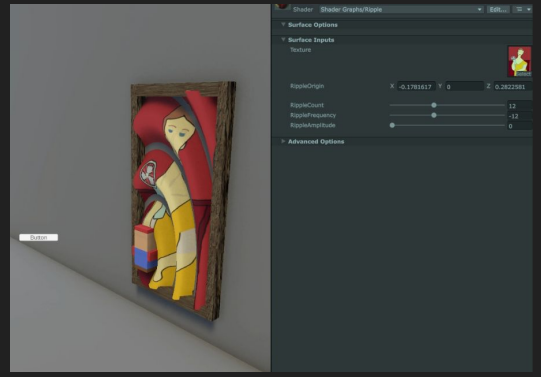
The photo to be recreated in Unity

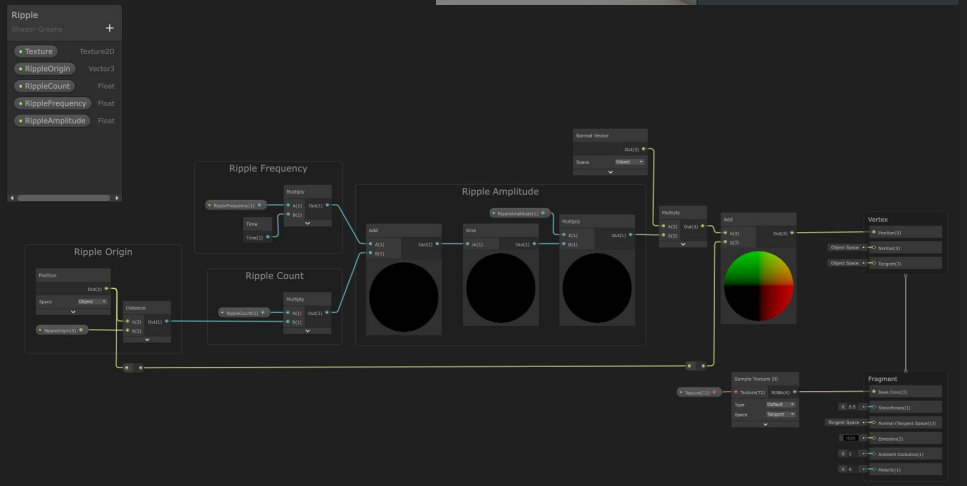


The Unity recreation of the photo

Exercise 4: Shader Graph

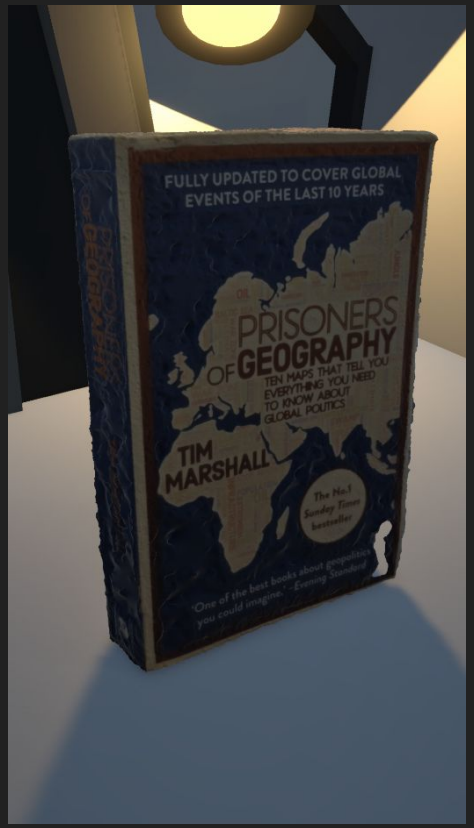
In this exercise Unity's node-based shader tool, Shader Graph, was used to make a ripple shader, inspired by the ripple effect when entering paintings in Super Mario 64. First the origin of the ripple is set in the object space, going into a Distance node with a Vector3 variable, RippleOrigin, that allows "Mario" to start ripples where he touches the painting. The Distance node is then multiplied with a RippleCount so the ripples are spread out over the mesh. To control the frequency a Time node is multiplied with a RippleFrequency variable. These to are added together to create the amplitude of the ripples through a Sine node multiplied with a RippleAmplitude variable.
In order to distort the mesh, the amplitude is multiplied with the normal, to make the ripples respect the geometry, in turn making the shader compatible with complex meshes. This is then added together with the position in the object space to finally distort the vertex position and generate the ripple effect.
This shader works best on meshes with a high poly count as it allows for smoother ripples.



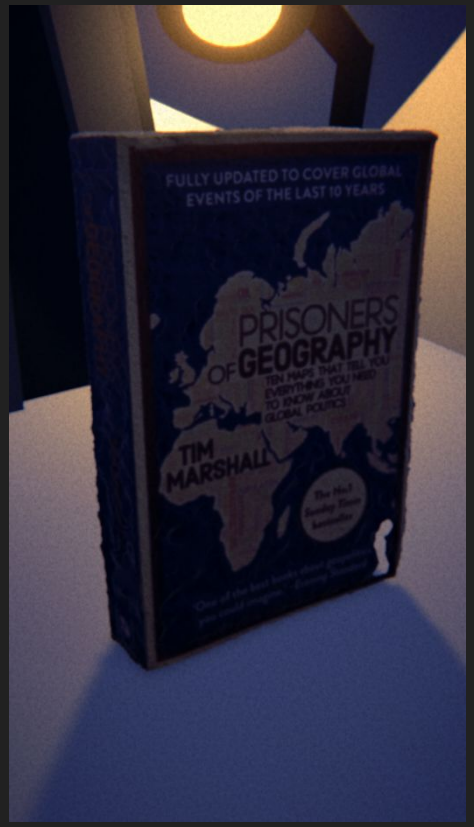


Exercise 5: Computer Graphics and Camera Effects

The same scene was used as from **Exercise 3** to play around with various lighting and camera effects in the Volume component, as were already done for that assignment. In this scene, the gamma was decreased and slightly tinted blue in order to create the a night atmosphere. The bulb of the lamp has a point light inside it as well as the material have color emission through bloom. The camera also have effects added to it to simulate various artifacts that typically appear in low-light environments such as grain, loss of focus and slight chromatic aberration near the edges and a discrete vignette for aesthetic purposes.



Without global volume effects



With global volume effects

Exercise 6: High-Level Shader Language (HLSL)

For this exercise a very simple wave shader was written in HLSL by using the Time variable and getting the sine of the mesh vertex multiplied by a given frequency. In this case the distortion is applied to the y position of the vertex, making the wave travel up and down. This can be changed to be sideways by setting the v.vertex.x to change, instead of v.vertex.y if that effect is desired. The function UnityObjectToClipPos is needed to transform the object's local vertex and transform them to clip space so they can be processed correctly. The TRANFORM_TEX function transforms the uv coordinates accordingly based on tiling and offset, ensuring the texture appears correctly. In order to make the shader more interesting, a texture can be applied and a color variable can affect it. In this case a smiley was used, and the waves can change the expression of the smiley marginally.

Again, like in **Exercise 4**, this shader gives the best results on a mesh with a high poly count as the edges will become smoother.

```
Shader "Custom/TestShader"
{
    Properties
    {
        _MainTex ("Texture", 2D) = "white" {}
        _Color ("Color", Color) = (1,1,1,1)
        _WaveAmplitude ("Wave Amplitude", Float) = 0.2
        _WaveSpeed ("Wave Speed", Float) = 1.0
        _WaveFrequency ("Wave Frequency", Float) = 2.0
    }

    SubShader
    {
        Tags { "RenderType"="Opaque" }

        Pass
        {
            CGPROGRAM
            #pragma vertex vert
            #pragma fragment frag

            #include "UnityCG.cginc"

            struct appdata
            {
                float4 vertex : POSITION;
                float2 uv : TEXCOORD0;
            };

            struct v2f
            {
                float2 uv : TEXCOORD0;
                float4 pos : SV_POSITION;
            };

            sampler2D _MainTex;
            float4 _MainTex_ST;
            float4 _Color;

            float _WaveAmplitude;
            float _WaveSpeed;
            float _WaveFrequency;

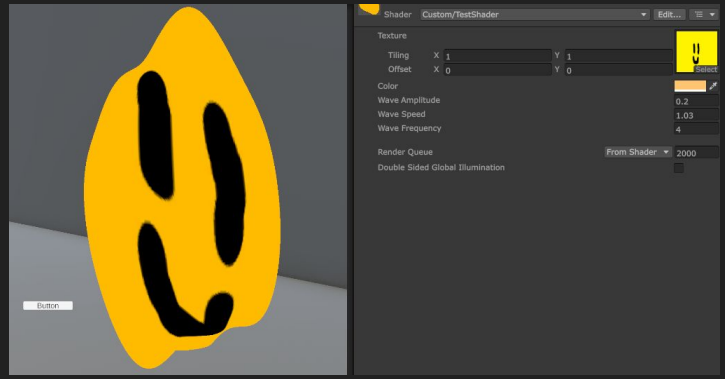
            v2f vert (appdata v)
            {
                // Initialize output structure
                v2f o;

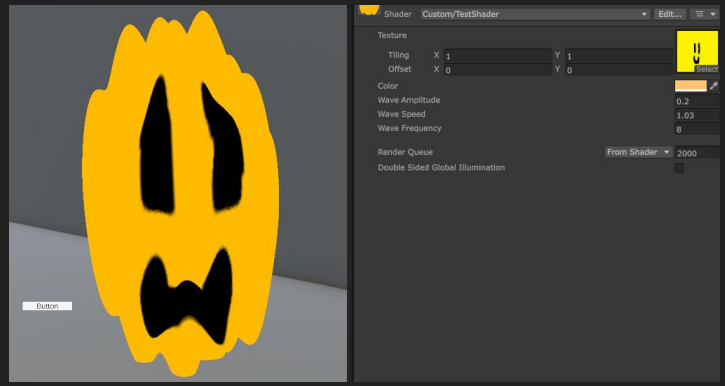
                // Calculate wave distortion based on time
                float time = _Time.y * _WaveSpeed;
                float wave =
                    sin(v.vertex.x * _WaveFrequency + time) *
                    sin(v.vertex.z * _WaveFrequency + time);

                // Apply wave distortion to vertex position
                v.vertex.y += wave * _WaveAmplitude;
                o.pos = UnityObjectToClipPos(v.vertex);
                o.uv = TRANSFORM_TEX(v.uv, _MainTex);

                return o;
            }

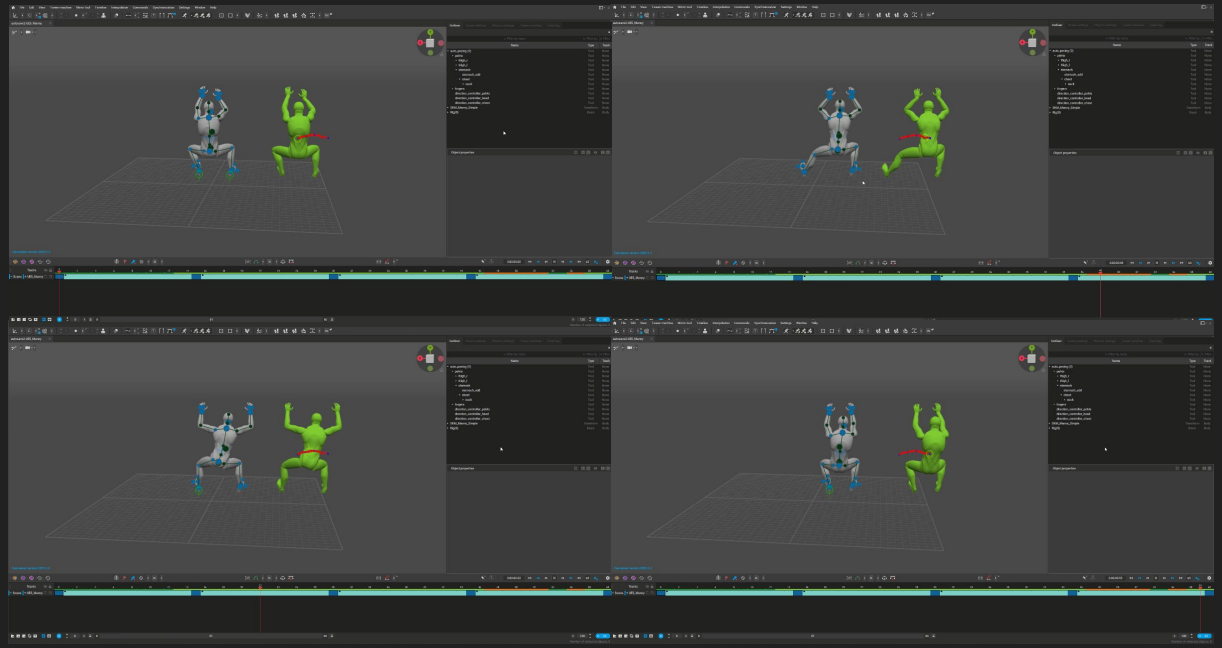
            fixed4 frag (v2f i) : SV_Target
            {
                // Sample texture
                fixed4 texColor = tex2D(_MainTex, i.uv);
                return texColor * _Color; // Get the color as well
            }
            ENDCG
        }
    }
}
```





Exercise 7: Real-time Animation

For this exercise the software Cascadeur was used to make an animation using keyframes and poses for a character. Cascadeur provides access to a library of multiple pre-rigged characters, meaning they have bones that are weighted accordingly and ready to be animated. I chose to go with a humanoid character, since the intent was to make an animation for another mini project. My animation would have the humanoid character hold on to a ledge and climb slideways. This was achieved with five keyframes with bezier interpolation in between them. Cascadeur also provides a wide array of options to tweak various interactions between the character and the physics of the world. These were not need for this animation's use case, so they were disabled.



Exercise 8: Tiny Virtual Production

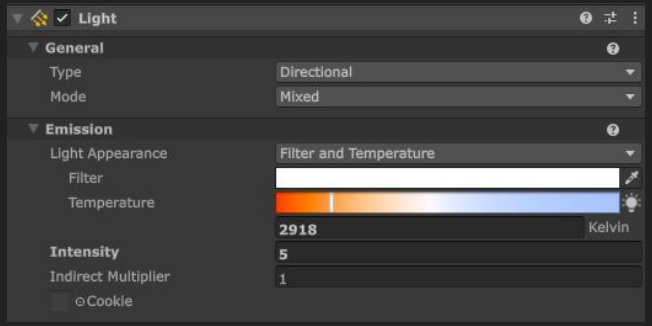
For this exercise the asset: Unity Terrain - URP Demo Scene was downloaded and used that as my virtual environment. I changed the camera to be at a static location and the lighting of the scene to be warmer. A warm light was chosen to make the scene look more cozy. For lighting the mouse actor, a lamp with color temperature control was used in an effort to match the virtual scenes lighting. I flicked my finger across the lamp to mimic moving leaves or branches above the actor casting a shadow on it.



The mouse actor in front of the screen that projects the virtual environment



The location of the shot in the virtual environment with the camera shown as a gizmo



The lighting settings of the virtual “sun”.

Exercise 9: Analysis of 3D Objects

The mesh that was created in Exercise 2 was used for this analysis exercise. The model was already simplified from RealityScan, but was still over 100 MB, which is totally unacceptable. Therefore I used PyVista to decimate the model to truly see how unoptimised it is. To do this, the decimate function was used to simplify the mesh the hard way - by straight up removing the faces. On the pictures, 80% of the faces are removed and even still you can still the the general shape of the mesh, which is a testament to how worrisome this 3D model actually is.



Another analysis was performed on the 3D model regarding the normals of the mesh. This was done by getting the normals from the point data and normalizing the values, adding a range from 0 to 1. The colors suggest that there are some inconsistencies in the mesh that possibly stem from the many faces that have sharp edges or other distortions in the geometry.

